



# CardiolA — Fase 7

## Plataforma de Inteligência Cardíaca Total

|             |                                                                               |
|-------------|-------------------------------------------------------------------------------|
| Instituição | FIAP — Faculdade de Informática e Administração Paulista                      |
| Disciplina  | ITIAOR2 — Inteligência Artificial                                             |
| Integrantes | Bruno Castro (RM558359) · Hugo Mariano (RM560688) · Matheus Castro (RM559293) |
| Repositório | github.com/Hugo345-l/cardioa-fiapi-fase7                                      |

### 1. Visão Geral

A **Fase 7** consolida a jornada da CardiolA — coleta de dados (Fase 1), diagnóstico automatizado (Fase 2), monitoramento IoT (Fase 3), análise de imagens (Fase 4), assistência conversacional (Fase 5) e orquestração preditiva multiagente (Fase 6) — em um **produto digital funcional, seguro e centrado no usuário**.

Se a Fase 6 entregou o "Cérebro" da solução (previsão de risco cardiovascular), a Fase 7 materializa o "Corpo": a interface que conecta médico e paciente aos insights de Inteligência Artificial em tempo real, integrando back-end, front-end, mobile, LLMs e sensores em uma única arquitetura orquestrada por Python.

### 2. Objetivo

Desenvolver o MVP final e funcional da plataforma CardiolA, capaz de:

- Realizar o **deploy completo**: Web (Vercel, com CI/CD ativo) e Mobile ( `.apk` via Expo EAS Build);
- **Integrar** os módulos das fases anteriores num back-end Python único (modelo preditivo da Fase 6 e LLM da Fase 5);
- **Refinar a Experiência do Usuário**, com foco no fluxo de informação dos indicadores de risco;
- **Evoluir o hardware**, migrando a lógica de sensores de C/C++ para MicroPython (ESP32).

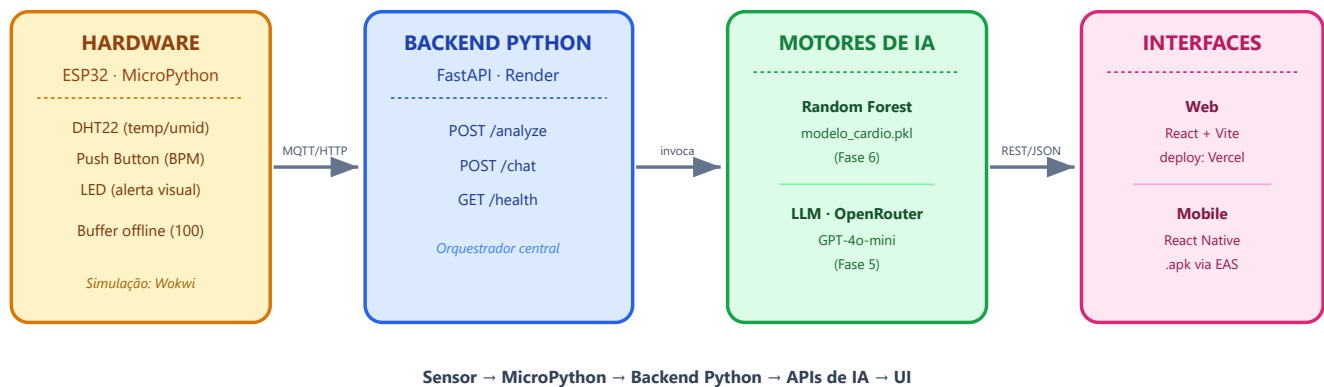
### 3. Funcionalidades da Plataforma

O profissional de saúde, através da aplicação Web ou Mobile, pode:

- **Monitorar** sinais vitais em tempo real (batimentos, temperatura, SpO<sub>2</sub>) no dashboard;
- **Analisar** o risco cardiovascular de um paciente a partir de 11 variáveis clínicas, com classificação em Alto / Médio / Baixo risco e protocolos clínicos sugeridos;
- **Consultar** um assistente conversacional especializado em cardiologia, alimentado por LLM;
- **Receber** dados de sensores IoT simulados em MicroPython.

## 4. Arquitetura Final da Solução

O fluxo de dados percorre quatro camadas, do sensor físico até a interface do usuário, orquestradas pelo back-end Python:



### 4.1. Descrição do Fluxo

- **Hardware (Edge):** o ESP32, programado em MicroPython, lê temperatura/umidade (DHT22) e simula batimentos (push button), acendendo um LED em condições críticas. Um buffer circular garante resiliência offline.
- **Backend Python (Orquestrador):** o serviço FastAPI recebe os sinais e expõe endpoints que acionam os motores de IA. É o ponto único de integração entre interfaces e modelos.
- **Motores de IA:** o endpoint `/analyze` carrega o modelo Random Forest treinado na Fase 6; o `/chat` consome o LLM via OpenRouter, herdando a assistência conversacional da Fase 5.
- **Interfaces (UI):** as aplicações Web (Vercel) e Mobile (APK) consomem a API via REST, entregando os insights ao usuário final.

**Decisão de arquitetura:** o back-end Python centraliza toda a lógica de IA, mantendo as interfaces "magras". Isso permite que Web e Mobile compartilhem exatamente a mesma fonte de verdade e que o modelo seja atualizado sem alterar os clientes.

## 5. Back-end Integrador e Motores de IA

O núcleo da solução é um serviço **FastAPI** (Python 3.12) que unifica as interfaces aos motores de IA das fases anteriores. Está hospedado no **Render.com**, com deploy automático via `render.yaml` (Blueprint).

### 5.1. Endpoints da API

| Método / Rota | Função                                                                                   | Origem |
|---------------|------------------------------------------------------------------------------------------|--------|
| GET /health   | Verifica status da API e carregamento do modelo                                          | —      |
| POST /analyze | Recebe 11 variáveis clínicas, retorna probabilidade, classificação de risco e protocolos | Fase 6 |
| POST /chat    | Recebe mensagem do usuário, retorna resposta cardiológica do LLM                         | Fase 5 |

### 5.2. Integração do Modelo Preditivo (Fase 6)

O endpoint `/analyze` carrega o artefato `modelo_cardio.pkl` — um **Random Forest Classifier** treinado na Fase 6 sobre ~70.000 registros do dataset *Cardiovascular Disease* (Kaggle). As 11 features (idade, sexo, altura, peso, pressões sistólica/diastólica, colesterol, glicose, tabagismo, álcool, atividade física) são montadas num `DataFrame` e submetidas a `predict_proba()`. A faixa de risco é derivada da probabilidade:

| Probabilidade | Classificação | Conduta                                           |
|---------------|---------------|---------------------------------------------------|
| ≥ 0,70        | ● Alto Risco  | Protocolos de urgência (ECG, encaminhamento <24h) |
| 0,40 – 0,69   | ● Médio Risco | Protocolos ambulatoriais (consulta <30 dias)      |
| < 0,40        | ● Baixo Risco | Orientações preventivas e rotina anual            |

Os protocolos clínicos correspondentes são consultados em `protocolos_medicos.json`, retornando uma lista de condutas por faixa de risco.

### 5.3. Integração do Assistente Conversacional (Fase 5)

O endpoint `/chat` conecta-se ao **OpenRouter** (modelo `gpt-4o-mini`) com um *system prompt* que delimita o assistente ao domínio da cardiologia, em português, sempre recomendando avaliação médica para questões específicas. A chave de API é injetada por variável de ambiente (`OPENROUTER_API_KEY`), nunca versionada.

**Segurança:** o CORS é habilitado para as origens dos clientes; credenciais sensíveis residem apenas nas variáveis de ambiente do Render, fora do controle de versão (`.gitignore`).

## 6. Front-end, Mobile e Hardware

### 6.1. Aplicação Web (React + Vite)

A interface Web é uma SPA construída em **React 18 + Vite**, com quatro telas e navegação protegida por sessão:

| Tela             | Função                                                                                |
|------------------|---------------------------------------------------------------------------------------|
| Login            | Autenticação de acesso à plataforma                                                   |
| Dashboard        | Monitoramento de sinais vitais em tempo real, com indicadores coloridos por gravidade |
| Análise de Risco | Formulário das 11 variáveis clínicas e exibição visual do resultado e protocolos      |
| Assistente       | Interface de chat com o LLM cardiológico                                              |

O deploy é feito na **Vercel**, com `vercel.json` configurando rotas SPA e **CI/CD ativo**: cada `git push` na branch `main` dispara um novo deploy automático.

### 6.2. Aplicação Mobile (React Native + Expo)

O app mobile espelha as funcionalidades da Web em **React Native + Expo**, com navegação em pilha. O `app.json` define o `android.package` em domínio invertido ( `br.com.cardioai.app` ) e o `eas.json` contém o perfil `preview` para gerar o `.apk` via **Expo EAS Build** (build na nuvem).

### 6.3. Hardware em MicroPython (Fase 3 evoluída)

A lógica de sensores da Fase 3, originalmente em **C/C++ (Arduino)**, foi convertida para **MicroPython**, permitindo execução em dispositivos mais flexíveis (ESP32 / Raspberry Pi). Principais equivalências:

| Arduino C/C++                       | MicroPython                       |
|-------------------------------------|-----------------------------------|
| <code>#include "DHT.h"</code>       | <code>import dht</code>           |
| <code>dht.readTemperature()</code>  | <code>sensor.temperature()</code> |
| <code>pinMode / digitalWrite</code> | <code>machine.Pin</code>          |
| <code>delay(ms)</code>              | <code>time.sleep_ms(ms)</code>    |
| <code>Serial.println()</code>       | <code>print()</code>              |

O script mantém o buffer circular de 100 leituras (resiliência offline) e adiciona feedback visual via LED para condições críticas (temperatura > 38 °C ou BPM > 120).

## 7. Deploy e Resultados Validados

Toda a cadeia foi testada de ponta a ponta em produção:

| Componente      | Plataforma          | Validação                                              |
|-----------------|---------------------|--------------------------------------------------------|
| Back-end (API)  | Render.com          | <code>/health</code> → <code>model_loaded: true</code> |
| Front-end (Web) | Vercel + CI/CD      | Fluxo login → dashboard → análise → chat OK            |
| Mobile (.apk)   | Expo EAS Build      | Build gerado com perfil <code>preview</code>           |
| IoT             | Wokwi (MicroPython) | Leitura de sensores e alerta por LED                   |

### 7.1. Testes Funcionais do Modelo

Validação real do endpoint `/analyze` em produção:

- **Paciente de alto risco** (55a, fumante, PA 160/100, colesterol muito alto): probabilidade **86,4%** → **Alto Risco**, com 8 protocolos de urgência.
- **Paciente saudável** (40a, PA 120/80, sem fatores de risco): probabilidade **16,7%** → **Baixo Risco**, com orientações preventivas.
- **Assistente LLM**: respostas coerentes e contextualizadas em cardiologia, em português.

## 8. Limitações e Próximos Passos

- A autenticação é simplificada (sessão local), adequada ao escopo de MVP acadêmico; uma versão de produção exigiria OAuth/JWT e persistência de usuários.
- O monitoramento do dashboard usa dados simulados; a integração física sensor→backend em tempo real é o passo natural seguinte.
- O free tier do Render hiberna após inatividade, gerando latência (~50s) na primeira requisição — contornável com plano pago ou *health-check* periódico.

## 9. Conclusão

A Fase 7 cumpriu o objetivo de transformar os módulos isolados das fases anteriores em um **ecossistema coeso e funcional**. A plataforma demonstra, na prática, como agentes de IA — preditivos e conversacionais — podem ser aplicados à saúde digital, respeitando boas práticas de engenharia (separação de camadas, segurança de credenciais, CI/CD) e entregando uma experiência acessível tanto na Web quanto no mobile. O resultado é um MVP que materializa o "Corpo" da CardiolA, conectando o profissional de saúde aos insights da Inteligência Artificial em tempo real.